Data Analytics Using SQL ANSWER BANK

Part A - 2 Marks

Q1. Define a relational database and state one advantage of using it for data analysis.

Introduction: A relational database is a structured collection of data organised into related tables, managed by a Relational Database Management System (RDBMS) such as MySQL, PostgreSQL, Oracle or SQL Server.

- **Definition:** Data is stored in tables (relations) made of rows (records) and columns (attributes). Tables are linked using primary and foreign keys, so every fact is stored once and referenced wherever it is needed.
- **Advantage for analysis:** Because data is organised, consistent and queryable with SQL, analysts can filter, join and summarise very large datasets quickly and accurately. For example, joining Customers and Orders tables gives total purchases per customer in one query.

Q2. Define SQL. Name any two sub-languages of SQL and give one command for each.

Introduction: SQL (Structured Query Language) is the standard declarative language used to create, manage and query data stored in relational databases.

- **Definition:** In SQL the user states what data is required and the database engine decides how to retrieve it. It is supported by all major RDBMS products, which makes it the most widely used language in data analytics.
- **Sub-languages:** DDL (Data Definition Language) defines table structure, e.g. CREATE TABLE Student(...). DML (Data Manipulation Language) works on the stored data, e.g. INSERT INTO Student VALUES (...). Others are DQL (SELECT), DCL (GRANT) and TCL (COMMIT).

Q3. Differentiate between the CHAR and VARCHAR data types with one example for each.

Introduction: CHAR and VARCHAR both store character strings but differ in how storage space is allocated, as compared below.

Basis	CHAR(n)	VARCHAR(n)
Length and storage	Fixed length: always occupies n characters and pads shorter values with spaces.	Variable length: stores only the actual characters plus a small length prefix, saving space.
Example and use	Code CHAR(2) holding 'IN'; best for fixed-size data such as country or grade codes.	Name VARCHAR(50) holding 'Amy West'; best for names, emails and addresses of varying size.

Q4. Write an SQL query to retrieve the RollNo and Name columns from a table named Student.

Introduction: The SELECT statement retrieves data from a table; listing column names after SELECT returns only those columns (projection).

- **Query:** Name the required columns after SELECT and the table after FROM.

```
SELECT RollNo, Name
FROM Student;
```

- **Working:** The database reads every row of Student and returns only the two listed columns. No WHERE clause is used, so all students appear. Writing * instead would return every column, which is unnecessary here.

Q5. Write an SQL query using WHERE to display the products whose Price is less than 500 from a Product table.

Introduction: The WHERE clause filters rows and returns only those that satisfy a given condition.

- **Query:** Use the comparison operator < in the WHERE condition.

```
SELECT *
FROM Product
WHERE Price < 500;
```

- **Working:** WHERE tests the condition on each row and keeps a row only when the condition is true. Conditions can be combined with AND, OR, NOT, BETWEEN or IN, e.g. WHERE Price BETWEEN 100 AND 500.

Q6. State the purpose of the DISTINCT keyword in SQL with an example.

Introduction: DISTINCT is a keyword used with SELECT to remove duplicate rows from the result set.

- **Purpose:** DISTINCT returns each unique value (or unique combination of values) only once. It affects only the query output and never changes or deletes data stored in the table.
- **Example:** The query below lists every department once, even if many employees belong to it. It is also used for unique counts, e.g. COUNT(DISTINCT Department).

```
SELECT DISTINCT Department
FROM Employee;
```

Q7. Name any two aggregate functions in SQL and state what each returns.

Introduction: Aggregate functions perform a calculation on a set of rows and return a single summary value.

- **COUNT():** Returns the number of rows (COUNT(*)) or the number of non-NULL values in a column. Example: SELECT COUNT(*) FROM Sales; returns the total number of sales records.
- **AVG():** Returns the arithmetic mean of a numeric column and ignores NULL values. Example: SELECT AVG(Salary) FROM Employee; returns the average salary of all employees.

Q8. What is a NULL value? Write a query to find the employees whose Department is NULL.

Introduction: NULL is a special marker in SQL that represents a missing, unknown or inapplicable value.

- **Meaning:** NULL is not zero and not an empty string. Any comparison such as Department = NULL evaluates to unknown, so NULL values can never be found with the = operator.
- **Query:** Use the IS NULL operator to test for missing values. COALESCE(Department, 'Unknown') can then replace them in the output.

```
SELECT *
FROM Employee
WHERE Department IS NULL;
```

Q9. What is data cleaning? Name one SQL function used to handle missing values.

Introduction: Data cleaning is the first step of data pre-processing and prepares raw data for reliable analysis.

- **Data cleaning:** It is the process of fixing or removing incorrect, incomplete, duplicate, corrupted or wrongly structured data in a dataset. Clean data prevents wrong but believable results from skewing business decisions.
- **COALESCE():** COALESCE(column, default) returns the first non-NULL value in its argument list. For example, COALESCE(Department, 'Unknown') shows 'Unknown' wherever Department is NULL, without altering the stored table.

Q10. State the purpose of the ORDER BY clause. Write a query to display employee names in alphabetical order.

Introduction: ORDER BY is the clause used to sort the rows of a query result.

- **Purpose:** ORDER BY sorts the result by one or more columns in ascending order (ASC, the default) or descending order (DESC). It changes only the order of the output, not the order in which data is stored.
- **Query:** The query sorts names from A to Z. For multiple columns, ORDER BY Department ASC, Salary DESC sorts by department first and then by highest salary.

```
SELECT Name
FROM Employee
ORDER BY Name ASC;
```

Part B - 5 Marks

Q11. Given a table Student(RollNo, Name, Course, Marks), write SQL queries to (i) retrieve all records, (ii) display only Name and Marks, (iii) list students whose Marks are greater than 75, and (iv) list the students of the course 'CSE' sorted by Marks in descending order.

Introduction: Retrieving data is the most common task in analytics and is done with the SELECT statement, whose basic form is SELECT columns FROM table WHERE condition ORDER BY column.

- **(i) Retrieve all records:** The asterisk (*) is a wildcard that selects every column of every row. It is useful for a quick look at a table but should be avoided in production queries because it fetches unnecessary data.

```
SELECT * FROM Student;
```

- **(ii) Display selected columns:** Listing column names after SELECT is called projection. Only the named columns appear in the output and the order of columns follows the order written in the query.

```
SELECT Name, Marks FROM Student;
```

- **(iii) Filter using WHERE:** WHERE checks each row against a condition and keeps only the rows for which it is true. Here only students scoring above 75 are returned.

```
SELECT * FROM Student
WHERE Marks > 75;
```

- **(iv) Filter and sort:** A WHERE condition can be combined with ORDER BY. The database first filters the CSE students and then sorts them so that the topper appears first.

```
SELECT * FROM Student
WHERE Course = 'CSE'
ORDER BY Marks DESC;
```

- **Order of processing:** The DBMS processes the clauses logically as FROM, then WHERE, then SELECT, then ORDER BY, even though SELECT is written first. Text values such as 'CSE' must be enclosed in single quotes, while numbers need none.

Q12. Explain SQL data types and the basic data manipulation commands in SQL with examples.

Introduction: A data type defines the kind of value a column can store, while data manipulation commands (DML) are used to add, modify, remove and read the data stored in tables.

- **Numeric data types:** INT stores whole numbers, DECIMAL(p,s) stores exact numbers with fixed decimal places and FLOAT stores approximate real numbers. They are used for columns on which calculations are performed, e.g. Salary DECIMAL(10,2).
- **Character data types:** CHAR(n) stores fixed-length text, VARCHAR(n) stores variable-length text and TEXT stores long passages. Text values are written inside single quotes, e.g. Name VARCHAR(50) holding 'Amy West'.
- **Date and time data types:** DATE stores values in YYYY-MM-DD format, while TIME and DATETIME store time and full timestamps. Storing dates as VARCHAR prevents date arithmetic, so functions such as STR_TO_DATE() are needed to convert them.

- **INSERT and UPDATE:** INSERT adds a new row to a table and UPDATE changes existing values, normally restricted by a WHERE clause. Omitting WHERE in an UPDATE changes every row, which is a common mistake.

```
INSERT INTO Employee (EmpID, Name, Salary)
VALUES (21, 'Ravi Kumar', 65000.00);
UPDATE Employee SET Salary = 70000 WHERE EmpID = 21;
```

- **DELETE and SELECT:** DELETE removes the rows that satisfy a condition, and SELECT only reads data without changing it. Always test the WHERE condition with SELECT before running DELETE.

```
DELETE FROM Employee WHERE EmpID = 21;
SELECT * FROM Employee;
```

Q13. For a table Faculty(FacultyID, Name, Department, Salary), write SQL queries to (i) sort the faculty by Salary in descending order, (ii) count the faculty in each Department, (iii) display the Department-wise average Salary, and (iv) show only those departments whose average Salary exceeds 60000.

Introduction: Sorting and grouping organise raw rows into meaningful summaries. ORDER BY arranges rows, whereas GROUP BY combines rows with the same value so that aggregate functions can be applied to each group.

- **(i) Sort by Salary:** ORDER BY with DESC arranges the highest salary first. Without DESC the default order is ascending, so the keyword is needed here.

```
SELECT * FROM Faculty
ORDER BY Salary DESC;
```

- **(ii) Count per department:** GROUP BY places all rows of one department into a single group and COUNT(*) counts the rows in each group. The output has one row per department.

```
SELECT Department, COUNT(*) AS TotalFaculty
FROM Faculty
GROUP BY Department;
```

- **(iii) Average salary per department:** AVG() is applied separately to each group created by GROUP BY. An alias (AS AvgSalary) gives the calculated column a readable name.

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Faculty
GROUP BY Department;
```

- **(iv) Filter groups with HAVING:** WHERE cannot use aggregate functions because it runs before grouping. HAVING filters the groups after aggregation, so it is used to keep only departments above the limit.

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Faculty
GROUP BY Department
HAVING AVG(Salary) > 60000;
```

- **Combining the clauses:** All clauses can be used together in the fixed order SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY. Every non-aggregated column in SELECT must also appear in GROUP BY, otherwise the query is invalid.

Q14. Differentiate between the WHERE and HAVING clauses with suitable examples.

Introduction: WHERE and HAVING are both filtering clauses, but WHERE filters individual rows before grouping, whereas HAVING filters groups after aggregation. The five main differences are given below.

No.	Basis	WHERE	HAVING
1	Definition	Filters individual rows of the table based on a condition.	Filters groups of rows produced by GROUP BY based on a condition.
2	Stage of execution	Applied before GROUP BY, directly after FROM.	Applied after GROUP BY and aggregation.
3	Works on	Single (raw) rows.	Groups or aggregated results.
4	Aggregate functions	Cannot be used, e.g. WHERE AVG(Salary) > 5 is invalid.	Can be used, e.g. HAVING AVG(Salary) > 60000.
5	Use without GROUP BY	Works with or without GROUP BY.	Normally used together with GROUP BY.

Example with WHERE:

```
SELECT Name, Salary FROM Employee
WHERE Salary > 50000;
```

Example with HAVING:

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 60000;
```

Q15. Explain the SQL constraints used to maintain data integrity and validity, with examples.

Introduction: Constraints are rules defined on table columns that the database enforces automatically, so that only valid, consistent data can be stored.

- **NOT NULL:** It forces a column to always contain a value. It is used for mandatory fields such as Name, so that incomplete records and missing values are prevented at the time of data entry.
- **UNIQUE:** It guarantees that no two rows have the same value in a column, e.g. Email. Unlike a primary key, a table can have several UNIQUE columns and (in most systems) they may allow one NULL.
- **PRIMARY KEY:** It uniquely identifies every row and combines the NOT NULL and UNIQUE rules. A table has only one primary key, e.g. EmpID in Employee, and it also prevents duplicate records.
- **FOREIGN KEY:** It is a column that refers to the primary key of another table and keeps the relationship valid. For example, Sales.ProductID must match an existing Product.ProductID, so sales of non-existent products are rejected.

- **CHECK and DEFAULT:** CHECK validates a value against a condition, e.g. Salary > 0, which is how range validation is enforced. DEFAULT supplies a value automatically when none is given. A combined example is shown below.

```
CREATE TABLE Employee (
  EmpID INT PRIMARY KEY,
  Name VARCHAR(50) NOT NULL,
  Email VARCHAR(80) UNIQUE,
  Salary DECIMAL(10,2) CHECK (Salary > 0),
  Department VARCHAR(30) DEFAULT 'Unassigned',
  DeptID INT,
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Q16. Explain the different types of SQL joins with suitable examples and a neat diagram.

Introduction: A JOIN combines rows from two or more tables based on a related column, usually a primary key and foreign key pair. The examples use Employee(EmpID, Name, DeptID) as table A and Department(DeptID, DeptName) as table B.

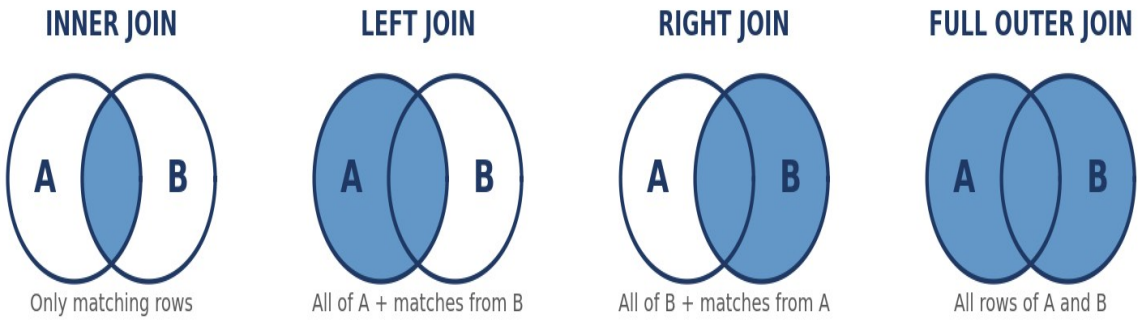


Fig. 1: Types of SQL joins (shaded area shows the rows returned)

- **INNER JOIN:** It returns only the rows that have matching values in both tables. Employees without a department and departments without employees are left out.

```
SELECT e.Name, d.DeptName
FROM Employee e
INNER JOIN Department d ON e.DeptID = d.DeptID;
```

- **LEFT JOIN:** It returns all rows of the left table and the matching rows of the right table; unmatched right-side columns are filled with NULL. It is used to find, for example, employees who have no department.

```
SELECT e.Name, d.DeptName
FROM Employee e
LEFT JOIN Department d ON e.DeptID = d.DeptID;
```

- **RIGHT JOIN:** It is the mirror image of LEFT JOIN: all rows of the right table and the matching rows of the left table are returned. It helps to list departments that currently have no employees.

```
SELECT e.Name, d.DeptName
FROM Employee e
RIGHT JOIN Department d ON e.DeptID = d.DeptID;
```

- **FULL OUTER JOIN:** It returns all rows from both tables, with NULL wherever there is no match. MySQL has no direct FULL JOIN, so it is simulated by taking the UNION of a LEFT JOIN and a RIGHT JOIN.

```
SELECT e.Name, d.DeptName
FROM Employee e
FULL OUTER JOIN Department d ON e.DeptID = d.DeptID;
```

- **CROSS JOIN and SELF JOIN:** CROSS JOIN pairs every row of one table with every row of the other (Cartesian product, m x n rows) and has no ON condition. A SELF JOIN joins a table with itself using aliases, e.g. to match employees with their managers.

Q17. Explain how data analytics supports decision-making in business and industry. Illustrate your answer with a suitable SQL-based example.

Introduction: Data analytics is the process of examining data to discover patterns and insights that support decisions. Since most business data sits in relational databases, SQL is the main tool used to turn it into information.

- **Fact-based decisions:** Analytics replaces guesswork with evidence taken from real transactions. Managers rely on summarised figures such as total revenue or order counts instead of intuition, which reduces the risk of wrong decisions.
- **Identifying trends and top performers:** Aggregation queries reveal which products, regions or periods perform best. For example, a retailer can rank categories by revenue and then focus stock and promotions on the top ones.

```
SELECT Category, SUM(Amount) AS TotalSales
FROM Sales
GROUP BY Category
ORDER BY TotalSales DESC;
```

- **Operational efficiency and cost control:** Analysing stock, delivery and production data highlights slow-moving items, delays and waste. Companies use these findings to cut inventory costs and improve processes.
- **Understanding customers:** Purchase history analysed with SQL helps segment customers, find repeat buyers and detect customers who may leave. This supports targeted offers and better customer service.
- **Industry applications:** Retail uses analytics for demand forecasting, banking for fraud detection and credit risk, healthcare for patient and resource planning, and manufacturing for quality control. In each case SQL extracts and prepares the data first.

Q18. What is a subquery in SQL? Explain its types with suitable examples.

Introduction: A subquery (inner query) is a SELECT statement nested inside another SQL statement. The inner query runs first and its result is used by the outer query.

- **Definition and syntax:** A subquery is written inside parentheses and can appear in the WHERE, FROM or SELECT clause. It lets one query depend on the result of another without creating temporary tables.
- **Single-row subquery:** It returns exactly one value and is used with operators such as =, > or <. The query below lists employees earning more than the average salary.

```
SELECT Name, Salary FROM Employee
WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

- **Multiple-row subquery:** It returns a list of values and is used with IN, ANY or ALL. The query below finds employees who work in departments located in Bengaluru.

```
SELECT Name FROM Employee
WHERE DeptID IN (SELECT DeptID FROM Department
                 WHERE Location = 'Bengaluru');
```

- **Correlated subquery:** The inner query refers to a column of the outer query and is re-evaluated for every outer row. It is used for row-by-row comparisons, such as employees earning above their own department's average.

```
SELECT Name, Salary FROM Employee e
WHERE Salary > (SELECT AVG(Salary) FROM Employee
                WHERE DeptID = e.DeptID);
```

- **Subquery in the FROM clause:** A subquery in FROM acts as a temporary derived table that must be given an alias. Compared with joins, subqueries are easier to read for step-by-step logic, but joins are often faster on large tables.

```
SELECT t.Department, t.AvgSal
FROM (SELECT Department, AVG(Salary) AS AvgSal
    FROM Employee GROUP BY Department) AS t
WHERE t.AvgSal > 60000;
```

Part C - 10 Marks

Q19. A company stores employee information in Employee(EmpID, Name, Department, Salary). Formulate a set of SQL queries to (i) retrieve the employees of a selected department, (ii) sort them by salary, (iii) find the number of employees, (iv) calculate the average, minimum and maximum salary, and (v) group the results by department. Explain the purpose of each query.

Introduction: Employee data is analysed with a small set of core SQL operations: filtering (WHERE), sorting (ORDER BY), aggregation (COUNT, AVG, MIN, MAX) and grouping (GROUP BY). Each query below answers one business question and is run on the sample data given in point 1.

- **Table design and sample data:** A table defines the structure of the data, with EmpID as the primary key so that every employee is unique. Five sample rows are inserted and used for all results below (IT: 80000 and 55000; Sales: 75000 and 72000; HR: 60000).

```
CREATE TABLE Employee (
  EmpID INT PRIMARY KEY, Name VARCHAR(50),
  Department VARCHAR(30), Salary DECIMAL(10,2));
INSERT INTO Employee VALUES
(1,'Amy West','HR',60000), (2,'Ivy Lee','Sales',75000),
(3,'Joe Smith','IT',80000), (4,'Jane Hill','IT',55000),
(5,'Dave West','Sales',72000);
```

- **(i) Employees of a selected department:** Filtering is the process of keeping only the rows that meet a condition. The WHERE clause selects the IT department, so HR can view only the employees under its concern. The result is Joe Smith and Jane Hill.

```
SELECT * FROM Employee
WHERE Department = 'IT';
```

- **(ii) Sort by salary:** Sorting arranges the result in a meaningful order. ORDER BY Salary DESC lists the highest paid employee first, which helps management to review the pay structure and identify top earners.

```
SELECT Name, Department, Salary
FROM Employee
ORDER BY Salary DESC;
```

- **(iii) Number of employees:** COUNT() is the aggregate function that counts rows. COUNT(*) counts all rows, so it gives the total headcount (5 for the sample data), which is used for workforce planning and budgeting.

```
SELECT COUNT(*) AS TotalEmployees
FROM Employee;
```

- **(iv-a) Average salary:** AVG() returns the arithmetic mean of a numeric column and ignores NULLs. It gives a benchmark salary (68400 for the sample data) against which individual salaries can be compared.

```
SELECT AVG(Salary) AS AvgSalary
FROM Employee;
```

- **(iv-b) Minimum and maximum salary:** MIN() and MAX() return the smallest and largest values of a column and can be used in the same query. Together they show the salary range of the company (55000 to 80000), which helps to detect pay gaps or outliers.

```
SELECT MIN(Salary) AS LowestSalary,
    MAX(Salary) AS HighestSalary
FROM Employee;
```

- **(v) Group by department:** GROUP BY divides the rows into groups of equal values and applies the aggregate functions to every group separately. The query gives one summary row per department (HR 1, IT 2, Sales 2).

```
SELECT Department,
    COUNT(*) AS Employees,
    AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Department;
```

Department	Employees	AvgSalary
HR	1	60000
IT	2	67500
Sales	2	73500

- **Filtering groups with HAVING:** HAVING is used to filter groups after aggregation, because WHERE cannot contain aggregate functions. The query keeps only the departments with an average salary above 65000, i.e. IT and Sales.

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employee
GROUP BY Department
HAVING AVG(Salary) > 65000;
```

- **Combining the clauses:** Several clauses can be used in one query as long as the fixed order SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY is followed. Logically, rows are filtered first, then grouped, then summarised and finally sorted.

```
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employee
WHERE Salary >= 55000
GROUP BY Department
ORDER BY AvgSalary DESC;
```

- **Overall purpose and interpretation:** Together these queries convert raw rows into information: who works where, what is paid, and how departments compare. Management can use the results for payroll budgeting, salary revision, recruitment planning and checking fairness of pay across departments.

Q20. A retail database contains Product(ProductID, ProductName, Category) and Sales(SaleID, ProductID, Quantity, Amount). Design SQL queries to join the tables, filter the sales above a specified amount, sort the results by Amount, and calculate the total sales amount and average sales amount for each Category. Explain the sequence of operations.

Introduction: Retail analysis needs data from more than one table, so the tables are first linked with a JOIN and then filtered, sorted and summarised. The two tables are related by ProductID, which is the primary key of Product and a foreign key in Sales (Fig. 2).



Fig. 2: Relationship between Product and Sales

- **Relationship between the tables:** A one-to-many relationship means one record of Product can be linked to many records of Sales. Product holds descriptive details once, while Sales stores each transaction, which avoids repeating product names in every sale.
- **Creating the tables and sample data:** The foreign key makes sure that a sale can only refer to an existing product. The sample data used in all results below has four products and six sales.

```
CREATE TABLE Product (ProductID INT PRIMARY KEY,
 ProductName VARCHAR(40), Category VARCHAR(30));
CREATE TABLE Sales (SaleID INT PRIMARY KEY, ProductID INT,
 Quantity INT, Amount DECIMAL(10,2),
 FOREIGN KEY (ProductID) REFERENCES Product(ProductID));
-- Products: 1 Laptop, 2 Phone (Electronics); 3 Shirt (Clothing); 4 Rice (Grocery)
-- Sales amounts: 60000, 50000, 4000, 800, 6000, 25000
```

- **Joining the tables:** INNER JOIN combines the rows of both tables where Product.ProductID equals Sales.ProductID. Table aliases (p, s) shorten the query and avoid ambiguity because both tables have a ProductID column.

```
FROM Sales s
INNER JOIN Product p ON s.ProductID = p.ProductID
```

- **Filtering sales above a specified amount:** The WHERE clause removes the rows that do not satisfy the condition, here sales not above 5000. The threshold can be changed to any specified amount, and this step also removes small, less relevant transactions.

```
WHERE s.Amount > 5000
```

- **Sorting by Amount (Query A):** ORDER BY s.Amount DESC arranges the filtered and joined rows from the highest amount to the lowest. Query A therefore gives a ranked detail list of high-value sales.

```
SELECT s.SaleID, p.ProductName, p.Category, s.Quantity, s.Amount
FROM Sales s
INNER JOIN Product p ON s.ProductID = p.ProductID
WHERE s.Amount > 5000
ORDER BY s.Amount DESC;
```

SaleID	ProductName	Category	Quantity	Amount
101	Laptop	Electronics	1	60000
102	Phone	Electronics	2	50000
106	Phone	Electronics	1	25000
105	Shirt	Clothing	3	6000

- **Total and average amount per category:** SUM(Amount) adds all sale amounts and AVG(Amount) gives the mean sale value of a group. They are aggregate functions, so they need a GROUP BY clause to produce one result per category rather than one overall value.
- **Grouping by Category (Query B):** GROUP BY p.Category forms one group for each category, and the aggregates are calculated for every group. The summary is sorted by total amount so that the best category comes first.

```
SELECT p.Category,
    SUM(s.Amount) AS TotalAmount,
    AVG(s.Amount) AS AvgAmount
FROM Sales s
INNER JOIN Product p ON s.ProductID = p.ProductID
WHERE s.Amount > 5000
GROUP BY p.Category
ORDER BY TotalAmount DESC;
```

Category	TotalAmount	AvgAmount
Electronics	135000	45000
Clothing	6000	6000

- **Refining the summary:** HAVING can filter categories after aggregation, e.g. HAVING SUM(s.Amount) > 10000, and ROUND(AVG(s.Amount), 2) makes the output tidy. HAVING is used here instead of WHERE because the condition is on an aggregate value.
- **Sequence of operations:** The database does not execute the clauses in the order in which they are written (Fig. 3). It first joins the tables (FROM/JOIN), then filters rows (WHERE), forms groups (GROUP BY), filters groups (HAVING), calculates the SELECT list and finally sorts (ORDER BY).
- **Interpretation and optimisation:** In the result, Grocery is missing because its only sale (800) was removed by the WHERE filter before grouping, so totals describe high-value sales only. Indexing the join column Sales.ProductID speeds up the join on large tables.

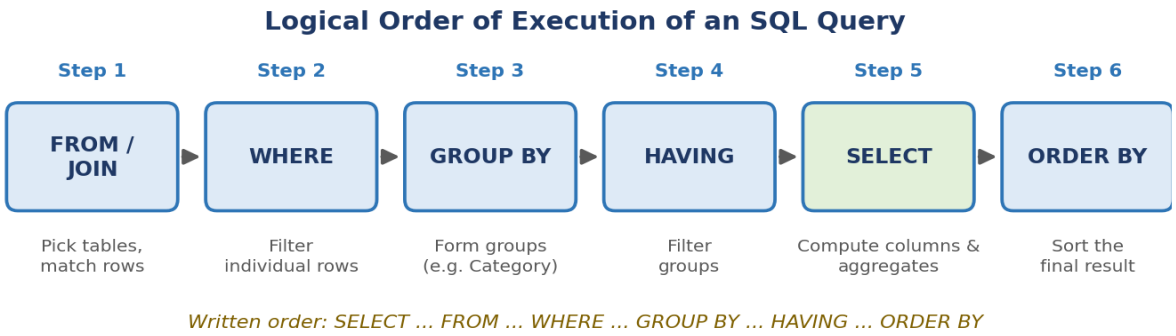


Fig. 3: Logical order in which the clauses of Query B are executed

Q21. A business wants to analyse its sales data using SQL. Using a Sales table containing SaleID, Product, Category, Quantity and Amount, develop an SQL-based analysis that retrieves relevant records, filters them using WHERE, groups them by Category, applies suitable aggregate functions and orders the resulting summary. State how the resulting information can support business analysis.

Introduction: Sales analysis converts transaction-level records into category-level performance figures. The workflow is: retrieve the relevant columns, filter out unwanted rows, group by Category, calculate aggregates and order the summary so that the most important results appear first.

- **Business objective and data:** A clear objective decides which columns and rows are relevant. Here the objective is to find which product categories generate the most revenue. The table below holds the eight sample records used in this answer.

SaleID	Product	Category	Quantity	Amount
1	Laptop	Electronics	2	90000
2	Mouse	Electronics	10	5000
3	Shirt	Clothing	5	4000
4	Jeans	Clothing	3	6000
5	Rice	Grocery	20	1600
6	Phone	Electronics	1	30000
7	Jacket	Clothing	2	5000
8	Oil	Grocery	10	1500

- **Retrieving relevant records:** Only the columns needed for the analysis are selected, which reduces the data processed. Product and SaleID are not required for the final summary, so Category, Quantity and Amount are retrieved.

```
SELECT Category, Quantity, Amount
FROM Sales;
```

- **Filtering with WHERE:** WHERE removes the records that should not take part in the analysis. Here transactions of 1500 or less (very small sales) are excluded, so the Oil sale (SaleID 8) is dropped before grouping.

```
SELECT Category, Quantity, Amount
FROM Sales
WHERE Amount > 1500;
```

- **Grouping by Category:** GROUP BY Category creates one group for each distinct category, so the aggregate functions work on each category separately. Without GROUP BY the aggregates would return a single value for the whole table.

```
... GROUP BY Category
```

- **Count and quantity totals:** COUNT(*) gives the number of transactions in each category and SUM(Quantity) the total units sold. These figures show the volume of business and the popularity of each category.

```
COUNT(*) AS Transactions,
SUM(Quantity) AS UnitsSold
```

- **Revenue measures:** SUM(Amount) gives total revenue, AVG(Amount) the average sale value and MAX(Amount) the largest sale. ROUND(AVG(Amount), 2) is used to keep the output readable.

```
SUM(Amount) AS TotalRevenue,
ROUND(AVG(Amount), 2) AS AvgSale,
MAX(Amount) AS LargestSale
```

- **Ordering the summary:** ORDER BY is applied last, on the grouped result, using the alias of an aggregate. Sorting TotalRevenue in descending order ranks the categories from the best to the weakest performer.

```
ORDER BY TotalRevenue DESC;
```

- **Complete analysis query:** All the parts are combined into one query that follows the clause order SELECT, FROM, WHERE, GROUP BY, ORDER BY. It produces one summary row per category.

```
SELECT Category,
    COUNT(*) AS Transactions,
    SUM(Quantity) AS UnitsSold,
    SUM(Amount) AS TotalRevenue,
    ROUND(AVG(Amount), 2) AS AvgSale,
    MAX(Amount) AS LargestSale
FROM Sales
WHERE Amount > 1500
GROUP BY Category
ORDER BY TotalRevenue DESC;
```

- **Output of the query:** The result shows that Electronics produces by far the highest revenue, followed by Clothing, while Grocery contributes the least. Grocery has high unit sales (20) but low revenue because its items are cheap.

Category	Transactions	UnitsSold	TotalRevenue	AvgSale	LargestSale
Electronics	3	13	125000	41666.67	90000
Clothing	3	10	15000	5000.00	6000
Grocery	1	20	1600	1600.00	1600

- **Support to business analysis:** Managers can use these results to decide on stock levels, marketing budgets and pricing: for example, protect the supply of Electronics, run offers on Clothing and bundle low-value Grocery items. Repeating the query monthly shows trends, helps in forecasting and measures whether decisions improved sales.

Q22. Explain the process of data cleaning and pre-processing in SQL. Discuss each step with suitable SQL queries.

Introduction: Data cleaning is the process of fixing or removing incorrect, incomplete, duplicate, corrupted or wrongly structured data. It is important because wrong data produces wrong but believable results. The process starts with data profiling and then follows the eight steps shown in Fig. 4. Queries are written for MySQL unless stated otherwise.

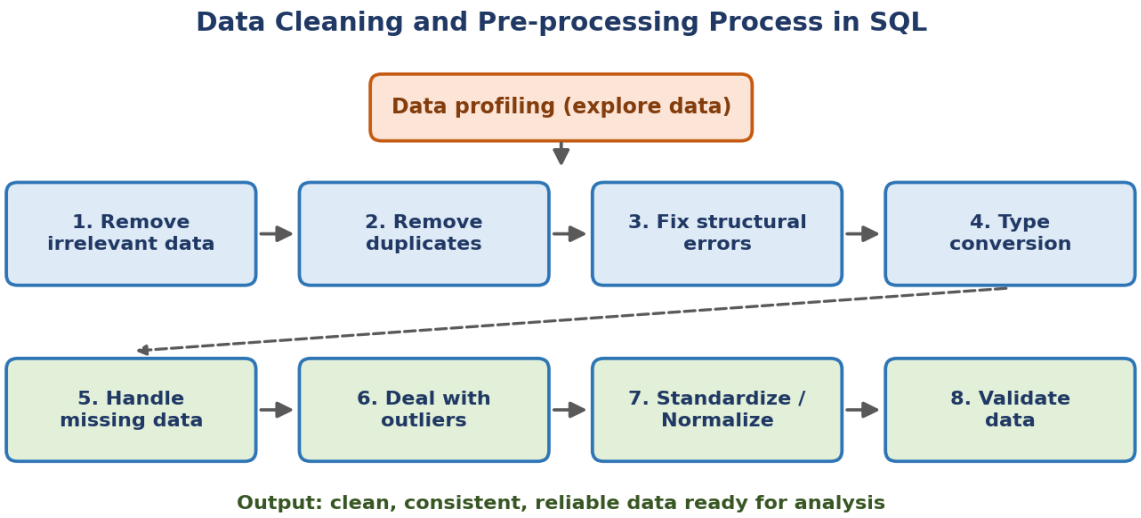


Fig. 4: Data cleaning and pre-processing process in SQL

- **Data profiling:** Profiling means exploring the structure, ranges and quality of the data before changing anything. The query counts rows, distinct values, NULLs and the minimum and maximum of a column to reveal problems.

```
SELECT COUNT(*) AS total_rows,
    COUNT(DISTINCT id) AS unique_ids,
    SUM(CASE WHEN year IS NULL THEN 1 ELSE 0 END) AS null_year,
    MIN(year) AS min_year, MAX(year) AS max_year
FROM customers;
```

- **Remove irrelevant data (Step 1):** Irrelevant data does not help the question being asked and can skew the results. If only US customers are being studied, customers of other countries are filtered out.

```
SELECT * FROM customers
WHERE country = 'US';
```

- **Remove duplicate data (Step 2):** Duplicate records give extra weight to some entries and distort analysis. SELECT DISTINCT * removes identical rows, while ROW_NUMBER() keeps only the latest record per id and is more efficient on wide tables.

```
SELECT * FROM (
 SELECT *, ROW_NUMBER() OVER (PARTITION BY id
                  ORDER BY year DESC) AS rn
 FROM customers) t
WHERE rn = 1;
```

- **Fix structural errors (Step 3):** Structural errors are typing mistakes, unusual naming, wrong capitalisation or extra spaces. TRIM() removes spaces, UPPER()/LOWER() fix case, REPLACE() corrects terms, and CASE converts different forms of the same value, such as N/A and Not Applicable, into NULL.

```
SELECT id, UPPER(TRIM(name)) AS name,
  CASE WHEN state IN ('N/A','Not Applicable')
     THEN NULL ELSE state END AS state_clean
FROM customers;
```

- **Type conversion (Step 4):** Type conversion changes values into the data type needed for calculation. Numbers stored as text, Unix timestamps and zip codes that lost their leading zeros are typical cases handled with CAST(), REPLACE(), LPAD() and FROM_UNIXTIME().

```
SELECT customer,
  LPAD(CAST(zipcode AS CHAR), 5, '0') AS zipcode,
  FROM_UNIXTIME(registered_at) AS registered_at,
  CAST(REPLACE(revenue, '$', '') AS UNSIGNED) AS revenue
FROM customers_zipcode;
```

- **Handle missing data (Step 5):** Missing values can be removed or replaced, depending on the context. Dropping rows is simple but loses information, whereas COALESCE() or IFNULL() replaces NULL with a default such as 'Unknown' or 0, or with a calculated value such as the average.

```
SELECT * FROM customers WHERE year IS NOT NULL;
SELECT COALESCE(department, 'Unknown') FROM employees;
```

- **Deal with outliers (Step 6):** Outliers are values far away from the rest and may be entry errors or genuine extremes. They are detected by a domain limit (e.g. salary above 100000) or a statistical rule such as the mean plus or minus 3 standard deviations, and then removed, capped or investigated.

```
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) + 3*STDDEV(salary) FROM employees)
  OR salary < (SELECT AVG(salary) - 3*STDDEV(salary) FROM employees);
```

- **Standardize and normalize (Step 7):** Standardizing brings data to a consistent unit, format or scale so that values can be compared. Examples are converting Fahrenheit to Celsius or a 5-point rating to a 100-point scale.

```
SELECT ROUND((fahrenheit - 32) * 5 / 9, 1) AS celsius
FROM temperatures;
SELECT points_5 * 20 AS points_100 FROM scores;
```

- **Validate data (Step 8):** Validation checks that the cleaned data follows the rules of the domain. Queries look for impossible values, such as negative salaries or a first-purchase year earlier than the store opened (2017), and a CHECK constraint prevents them in future.

```
SELECT * FROM employees WHERE salary < 0 OR salary > 100000;
SELECT * FROM customers WHERE year < 2017;
ALTER TABLE employees ADD CONSTRAINT chk_sal CHECK (salary > 0);
```

- **Deriving columns and best practice:** Derived columns such as years of service are calculated from existing columns to make analysis easier. In practice, users often have only read access, so cleaning is done in SELECT queries or views and does not change the original table.

```
SELECT employee_id, hire_date,
 TIMESTAMPDIFF(YEAR, hire_date, CURDATE()) AS years_of_service
FROM employees;
```

Q23. Explain the purpose of aggregate functions in SQL. Demonstrate the use of COUNT(), SUM(), AVG(), MIN() and MAX() on a Sales table along with the GROUP BY and HAVING clauses.

Introduction: Aggregate functions summarise many rows into a single value and are the foundation of reporting and analytics in SQL. The examples use the Sales(SaleID, Product, Category, Quantity, Amount) table with the eight sample rows of Q21: Amount totals 143100 and the categories are Electronics, Clothing and Grocery.

- **Purpose of aggregate functions:** An aggregate function takes a set of values and returns one summary value, such as a count, total or average. They are used to answer questions such as how many, how much and what is the highest, and they ignore NULL values (except COUNT(*)).
- **COUNT():** COUNT(*) counts all rows, COUNT(column) counts non-NULL values and COUNT(DISTINCT column) counts unique values. For the sample data the query below returns 8 sales and 3 distinct categories.

```
SELECT COUNT(*) AS TotalSales,
    COUNT(DISTINCT Category) AS Categories
FROM Sales;
```

- **SUM():** SUM() adds the values of a numeric column. It is used for totals such as revenue and units sold. The total revenue of the sample data is 143100 and the total quantity is 53.

```
SELECT SUM(Amount) AS TotalRevenue,
    SUM(Quantity) AS TotalUnits
FROM Sales;
```

- **AVG():** AVG() returns the mean of a numeric column, calculated as SUM divided by the number of non-NULL values. The average sale value here is 143100 / 8 = 17887.50.

```
SELECT AVG(Amount) AS AvgSale
FROM Sales;
```

- **MIN():** MIN() returns the smallest value of a column and works on numbers, text and dates. It identifies the lowest sale (1500) or the earliest date in a table.

```
SELECT MIN(Amount) AS SmallestSale
FROM Sales;
```

- **MAX():** MAX() returns the largest value of a column. It identifies the biggest sale (90000) or the latest date. MIN and MAX together show the range of the data and help to spot outliers.

```
SELECT MAX(Amount) AS LargestSale
FROM Sales;
```

- **NULL handling:** Aggregate functions skip NULL values, so COUNT(Amount) can be smaller than COUNT(*) and AVG(Amount) is calculated only over the non-NULL rows. If a NULL should be treated as zero, use AVG(COALESCE(Amount, 0)).

- **GROUP BY with aggregates:** GROUP BY splits the rows into groups of the same value and the aggregate functions are calculated for each group. Every column in SELECT that is not inside an aggregate function must appear in GROUP BY.

```
SELECT Category,
    COUNT(*) AS Sales, SUM(Amount) AS Total,
    AVG(Amount) AS Average,
    MIN(Amount) AS Lowest, MAX(Amount) AS Highest
FROM Sales
GROUP BY Category;
```

Category	Sales	Total	Average	Lowest	Highest
Electronics	3	125000	41666.67	5000	90000
Clothing	3	15000	5000.00	4000	6000
Grocery	2	3100	1550.00	1500	1600

- **HAVING clause:** HAVING filters the groups after aggregation, while WHERE filters rows before grouping. Aggregate functions are allowed only in HAVING, so the query below keeps categories with revenue above 10000 (Electronics and Clothing).

```
SELECT Category, SUM(Amount) AS Total
FROM Sales
GROUP BY Category
HAVING SUM(Amount) > 10000;
```

- **Combined query and rules:** WHERE, GROUP BY, HAVING and ORDER BY can be used together and are processed in that logical order. Remember that aggregates cannot be used in WHERE and that the HAVING condition can refer to an aggregate expression.

```
SELECT Category, SUM(Amount) AS Total
FROM Sales
WHERE Quantity > 1
GROUP BY Category
HAVING SUM(Amount) > 10000
ORDER BY Total DESC;
```

Q24. Define data analytics. Explain its types and lifecycle, and discuss the role of SQL in the data analytics process.

Introduction: Data analytics is the science of examining raw data to find patterns, draw conclusions and support decisions. It follows a lifecycle from collecting data to taking action, shown in Fig. 5, and SQL plays a central role in the storage, preparation and analysis stages.

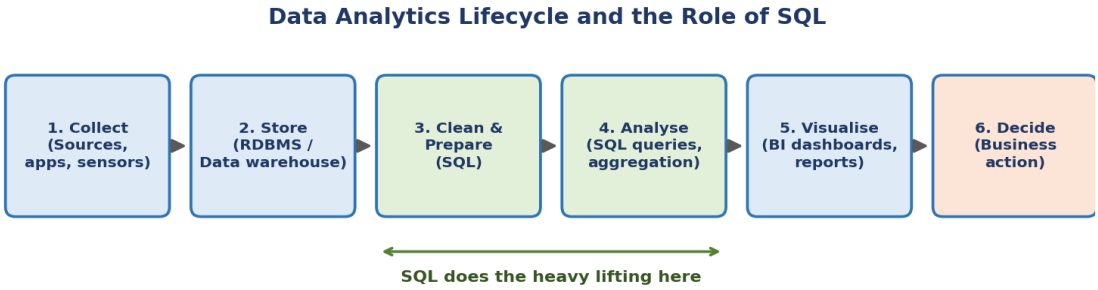


Fig. 5: Data analytics lifecycle and the role of SQL

- **Definition of data analytics:** Data analytics is the process of collecting, cleaning, transforming and analysing data to extract useful information. Its aim is to turn raw data into insight, so that organisations can take better decisions in areas such as sales, finance, healthcare and operations.
- **Descriptive analytics:** It answers the question what happened by summarising historical data. Sales reports, monthly totals and averages are examples, and most of them are produced directly with SQL aggregate queries such as SUM() and COUNT().
- **Diagnostic analytics:** It answers the question why it happened by drilling into the data to find causes. For example, grouping sales by region and product can show why revenue fell in a particular quarter. Joins, filters and GROUP BY are widely used.
- **Predictive analytics:** It answers the question what is likely to happen by applying statistical and machine learning models to past data. Examples are demand forecasting and customer churn prediction. SQL prepares the historical training data for these models.
- **Prescriptive analytics:** It answers the question what should be done by recommending the best action using prediction, rules and optimisation. Examples are suggesting stock quantities or the best price. It depends on accurate and clean data prepared earlier.
- **Stages of the lifecycle:** The lifecycle consists of collecting data from sources, storing it in a database or warehouse, cleaning and preparing it, analysing it, visualising the results and finally deciding on an action. The process repeats as new data arrives and decisions are reviewed.
- **SQL for storage and retrieval:** SQL creates the tables that store data and retrieves exactly the required rows and columns with SELECT, WHERE and JOIN. Since most organisations keep data in relational databases and warehouses, SQL is the common way to access it, e.g. SELECT Name, Department FROM Employee WHERE Salary > 50000;
- **SQL for cleaning and transformation:** Data pipelines (ETL or ELT) use SQL to clean and transform data, for example with COALESCE(), TRIM(), CAST() and CASE. Tools such as dbt let teams define the whole transformation process inside the warehouse using SQL.
- **SQL for analysis and reporting:** Aggregate functions, GROUP BY, window functions and subqueries calculate totals, averages, rankings and trends. The query results feed BI tools such as Power BI or Tableau, which present them as dashboards and reports.
- **Strengths and limitations:** SQL is fast on large structured data, easy to learn, standard across databases and efficient in cloud warehouses. Its limitations are that it is weaker for advanced statistics, machine learning and unstructured data, where it is combined with Python or R.